Execution Environments

Step by step: Windows, AD and SQL - Network (Cisco and Palo Alto) - and one EE that has it all

Build them, check them, add them to AAP | AAP 2.7, ansible-builder 3 | October 2026

What this is. A step-by-step guide to building three execution environments (EEs) for Ansible Automation Platform 2.7: one for **Windows, Active Directory and SQL** (SQL Server, MySQL, MariaDB), one for the **network** (Cisco and Palo Alto), and an **all-in-one** EE that has everything. Each one is built on top of **your base EE,** `base-ee:1.0`, and adds to everything it has. You build each with `ansible-builder` and podman, check it with one command, and add it to AAP. Every step says what to run and what you should see.

How to read it
Chapter 1 explains the three EEs. Do chapter 2 **once**. Then do the chapter of each EE you want (3, 4 or 5), and chapter 6 for each one you built. Chapter 8 lists what to do when something goes wrong. The build files are in the appendix, and in the repository's `ee/` folder. Rendered from `docs/EXECUTION_ENVIRONMENT.md` and `ee/` in the site-automation repository (version 0.32.0).

Host names and addresses in this guide are placeholders. Copy the files from the repository's ee/ folder, not from the PDF.

Contents

1. Start here 4

How the guide works . . . 4

Where each piece comes from . . . 5

2. Once, before any EE 6

Step 1. The build machine . . . 6

Step 2. Put the build files on the machine . . . 6

Step 3. Your base EE: base-ee:1.0 . . . 7

Step 4. Point the builds at your automation hub . . . 8

Step 5. Your hub token . . . 9

Step 6. Your CA certificates . . . 9

Step 7. Check that the build can reach PyPI . . . 9

3. EE 1: Windows, Active Directory and SQL 11

What is in it . . . 11

Step 1. Go to the folder . . . 11

Step 2. Build it . . . 12

Step 3. Check it . . . 12

Step 4. Check Kerberos to Active Directory . . . 12

Step 5. Put it into AAP . . . 12

Step 6. A first job . . . 12

Active Directory as an AAP inventory . . . 15

4. EE 2: Network - Cisco and Palo Alto 17

What is in it . . . 17

Step 1. Go to the folder . . . 17

Step 2. Build it . . . 17

Step 3. Check it . . . 18

Step 4. Put it into AAP . . . 18

Step 5. Cisco: a first job . . . 18

Step 6. Palo Alto: a first job . . . 18

5. EE 3: All-in-one 20

What is in it . . . 20

Step 1. Go to the folder . . . 20

Step 2. Build it . . . 21
Step 3. Check it . . . 21
Step 4. Put it into AAP . . . 21
Step 5. A first job . . . 21

6. Put an EE into AAP **22**

Step 1. Put the image where AAP can pull it . . . 22
Step 2. Add it in AAP . . . 22
Step 3. Use it . . . 22

7. Keep them up to date **24**

Pin the versions after the first good build . . . 24
Rebuild with a new tag . . . 24
Why three collections are pinned . . . 24
base-ee:1.0 already has some of these collections . . . 24
A new base EE . . . 24

8. When something goes wrong **25**

9. Appendix: the build files **27**

EE 1: Windows, AD and SQL - folder windows/ . . . 27
EE 2: Network (Cisco and Palo Alto) - folder network/ . . . 31
EE 3: All-in-one - folder all-in-one/ . . . 34
The same in every folder . . . 39

1. Start here

An **execution environment (EE)** is the container image an AAP job runs in. It holds Ansible, the collections (the modules) and the Python libraries those modules need. A job can only use what its EE has. If a module is missing, the job fails with `couldn't resolve module/action`.

This guide builds three EEs, step by step. Each one starts from **your base EE,** `base-ee:1.0`, the one you built with the earlier EE guide. It keeps everything `base-ee:1.0` has and adds more. None of them is built on Red Hat's bare-minimum image.

EE	Image name	What it is for
Windows, AD and SQL	`site-ee-windows`	Windows servers over WinRM (Kerberos), Active Directory (and AD as an AAP inventory), SQL Server, MySQL and MariaDB
Network	`site-ee-network`	Cisco (IOS / IOS-XE, Nexus, ASA) and Palo Alto (PAN-OS firewalls and Panorama)
All-in-one	`site-ee-all`	Everything above, plus Linux, VMware, NetApp, ServiceNow and every site-automation job

Which ones do I build?
You can build all three, or only the ones you need. The all-in-one EE alone is enough for every job. The other two are smaller and focused: a Windows team or a network team gets only what it uses. The steps are the same for each EE.

How the guide works

- **Chapter 2** you do **once**: the build machine, the logins, the files. It covers all three EEs.
- **Chapters 3, 4 and 5** are one EE each. Do only the ones you want, in any order.
- **Chapter 6** puts an EE into AAP. Do it for each EE you build.
- Every step has a command to run and a **You should see** box. If you see something else, stop and look it up in chapter 8, *When something goes wrong*.
- Text like `aap.example.mil` or `EXAMPLE.MIL` is a placeholder. Use your own names.

Where each piece comes from

Piece	Comes from	Supported by
Your base EE, `base-ee:1.0`: RHEL 9, Python, ansible-core, ansible-runner and what you added to it	your build, on top of Red Hat's AAP image from `registry.redhat.io`	Red Hat, for the Red Hat parts
System packages (`bindep.txt`: `kinit`, `dig`, `ssh` ...)	Red Hat's RHEL repositories, through the build machine's subscription	Red Hat
Certified collections (`ansible.windows`, `microsoft.ad`, `cisco.ios` ...)	Red Hat Automation Hub, or your Private Automation Hub	Red Hat (partner collections: the vendor)
Community collections (`community.general`, `lowlydba.sqlserver` ...)	Ansible Galaxy, or your hub's *community* repository	the community
Python libraries (`pywinrm`, `pymssql`, `pan-os-python` ...)	**PyPI** (pypi.org), or your PyPI mirror	their authors, **not** Red Hat

The Python libraries are not RPMs. Red Hat does not sign or patch them, and a RHEL update does not change them. They change only when you rebuild the EE.

Tested, and what was not

All three EEs were built from these exact files in October 2026 with ansible-builder 3.2. They were built on top of a stand-in for `base-ee:1.0`: an EE built on Red Hat's public UBI 9 minimal image, with ansible-core 2.16, Python 3.12, and older Windows, AD and network collections already in it. `ee-check` said `ALL YES` for each EE. The collections the base already had were kept, except `ansible.windows`: it went from 2.8 to 3.9 because `community.windows` needs it. An earlier build of the same EEs on ansible-core 2.20 passed too. Every example playbook in chapters 3 and 4 passed a syntax check, and its modules got as far as connecting. All 60 site-automation playbooks passed a syntax check in the all-in-one EE. **Not tested:** your real `base-ee:1.0` and your hub. The collections came from Galaxy, so `redhat.rhel_system_roles` was left out. Your first build is that test, and chapter 8 has what to do if something differs.

2. Once, before any EE

Do this chapter one time. Everything in it is used by all three EEs.

Step 1. The build machine

You need a **RHEL 9** machine with a RHEL subscription. The AAP server works, or any RHEL 9 server that can reach your automation hub. Log in as a normal user (not root).

```shell
sudo dnf install -y podman ansible-builder unzip
ansible-builder --version
podman --version
```

`ansible-builder` comes from the AAP repository. The machine needs the *Ansible Automation Platform* subscription for it. Without one: `pip3 install --user ansible-builder`.

You should see
`ansible-builder --version` prints `3.` something, and `podman --version` prints a version. Both commands are found.

Step 2. Put the build files on the machine

The build files are in the repository's `ee/` folder, one folder per EE - and in every release zip. Copy the release zip (or the `ee/` folder of your clone) to the build machine, then:

```shell
cd ~ && unzip -q site-automation-0.32.0.zip 'site-automation-0.32.0/ee/*'   # your version
mv site-automation-0.32.0/ee ~/ee && rmdir site-automation-0.32.0
cd ~/ee && ls
```

You should see
`all-in-one network windows`

Each folder has the same kinds of files:

File	What it holds	Do you change it?
`execution-environment.yml`	the recipe: the base image, the lists below, the build steps	only the base image's name, if yours differs (step 3)
`requirements.yml`	the collections	only to add or remove one
`requirements.txt`	the Python libraries (from PyPI) that no collection lists	only to add one
`bindep.txt`	the RHEL packages that no collection lists	only to add one
`ansible.cfg.example`	where the collections come from: your hub	you copy it to `ansible.cfg` (step 4)

File	What it holds	Do you change it?
`ca/`	your CA certificates	you put them in (step 5)
`check-ee.sh`, `check-list.txt`	`ee-check`: a test that runs inside the finished image	no
`krb5-ad-enctypes.conf`	Kerberos to Active Directory from a FIPS host (Windows and all-in-one only)	no

Step 3. Your base EE: base-ee:1.0

Every EE in this guide starts from `base-ee:1.0`, the EE you built with the earlier EE guide. The new EEs keep everything it has and add to it. The build machine needs that image:

```shell
podman images | grep base-ee
```

You should see
a line `localhost/base-ee 1.0 ...`.

Not there? Get it from wherever you keep it:

```shell
podman login aap.example.mil
podman pull aap.example.mil/base-ee:1.0       # from your hub's registry
podman load -i base-ee-1.0.tar                # or from a file you saved
```

If you pulled it from the registry, give it the local name too. Then everything in this guide works as written, with no file to change:

```shell
podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0
```

See what it has. These three commands show its ansible-core version, the Python that Ansible runs on, and its collections:

```shell
podman run --rm localhost/base-ee:1.0 ansible --version
podman run --rm localhost/base-ee:1.0 bash -c 'head -1 $(command -v ansible)'
podman run --rm localhost/base-ee:1.0 ansible-galaxy collection list
```

You should see
`ansible [core 2.16...]` or `ansible [core 2.20...]`; both work with these files. Then `#!/usr/bin/python3.12`, and then the list of collections it already has.

The Python must match. The three `execution-environment.yml` files say `python_path: /usr/bin/python3.12`. If the second command shows another one, for example `#!/usr/bin/python3.11`, put that path in the three files:

```shell
cd ~/ee
sed -i 's|/usr/bin/python3.12|/usr/bin/python3.11|' */execution-environment.yml
```

No `base-ee:1.0`? Then use AAP's **Default** execution environment, Red Hat's `ee-supported-rhel9`, which already holds Red Hat's supported collections. Do not use the *Minimal* one, which is the bare minimum. Put this name in the three files instead:

```
registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
```

You need to log in to Red Hat's registry first, with your Red Hat account or a registry service account: `podman login registry.redhat.io`.

Step 4. Point the builds at your automation hub

The collections are downloaded from your automation hub during the build. Make `ansible.cfg` once, then copy it into each folder:

```shell
cd ~/ee
cp windows/ansible.cfg.example windows/ansible.cfg
vi windows/ansible.cfg
```

In `ansible.cfg`, change `aap.example.mil` to your hub's address in the three `url =` lines. AAP shows the right addresses: **Automation Content > Repositories >** a repository **> Copy CLI configuration**. Then copy the file to the other two folders:

```shell
cp windows/ansible.cfg network/ansible.cfg
cp windows/ansible.cfg all-in-one/ansible.cfg
```

> **You should see**
> `cat network/ansible.cfg` shows your hub's address, not `aap.example.mil`.

No Private Automation Hub? Use console.redhat.com for the certified collections and Galaxy for the community ones. Replace the whole `ansible.cfg` with:

```
[galaxy]
server_list = rh_certified, galaxy

[galaxy_server.rh_certified]
url = https://console.redhat.com/api/automation-hub/content/published/
auth_url = https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token

[galaxy_server.galaxy]
url = https://galaxy.ansible.com/
```

Keep the name `galaxy` for the Galaxy server. Galaxy needs no token, and it refuses one that is not its own (`HTTP Code: 401`).

Tell your hub admin that these collections must be in the hub. The certified ones are in the *rh-certified* repository. The ones below come from Galaxy, so the hub's *community* repository has to sync them:

- `community.general`, `community.windows`, `community.crypto`, `containers.podman`
- `lowlydba.sqlserver`
- `paloaltonetworks.panos` and `ansible.mysql`, if your *rh-certified* repository does not already have them

If a collection cannot be had, remove its line from `requirements.yml` and `check-list.txt`.

Step 5. Your hub token

The build needs a token to download from your hub. Get it in AAP: **Automation Content > API token > Load token**. For console.redhat.com: **console.redhat.com > Automation Hub > Connect to Hub > Load token**.

The token never goes in a file. Paste it into the shell when you build:

```shell
read -rs TOKEN
```

Paste the token, then press Enter. Nothing shows on the screen, and the token stays out of the shell history.

> **You should see**
> `echo ${#TOKEN}` prints a number bigger than 0: the token's length.

`TOKEN` lasts until you close the shell. In a new shell, run `read -rs TOKEN` again.

Step 6. Your CA certificates

Put your root and intermediate CA certificates into each folder's `ca/`, one `.pem` file each. A PEM file is text that starts with `-----BEGIN CERTIFICATE-----`. The build adds them to the EE's trust store, so TLS to LDAPS, vCenter, Panorama, ServiceNow and AAP is checked against your CA.

```shell
cd ~/ee
for d in windows network all-in-one; do cp ~/my-ca/*.pem $d/ca/; done
ls windows/ca
```

> **You should see**
> your `.pem` files, next to `README.txt`.

No CA files? The build still works. You just will not have your CA in the EE.

Step 7. Check that the build can reach PyPI

The Python libraries come from PyPI, pypi.org:

```shell
curl -sI https://pypi.org/simple/pip/ | head -1
```

> **You should see**
> `HTTP/2 200`, or `HTTP/1.1 200 OK`.

No answer? Your site needs a PyPI mirror, such as Nexus or Artifactory. Get its address from whoever runs it, for example `https://nexus.example.mil/repository/pypi/simple`. When you build, add this line to the `podman build` command, just before the line `-f context/Containerfile`:

```
  --build-arg PIP_INDEX_URL=https://nexus.example.mil/repository/pypi/simple \
```

You are ready. Go to the chapter of the EE you want.

3. EE 1: Windows, Active Directory and SQL

Image name: `site-ee-windows`. Folder: `~/ee/windows`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, plus:

For	Collections	Python libraries and tools
Windows	`ansible.windows`, `community.windows`	`pywinrm` with Kerberos, `pypsrp`, `requests-kerberos`, `requests-ntlm`; `kinit`, `klist`
Active Directory	`microsoft.ad`	`pyspnego`, `gssapi`, `krb5`, `sansldap`, `ldap3`, `dnspython`; `dig`
SQL Server	`community.general` (`mssql_db`, `mssql_script`), `lowlydba.sqlserver`	`pymssql`
MySQL and MariaDB	`ansible.mysql`	`PyMySQL`
The report website on an SMB share		`smbprotocol` (Kerberos through `pyspnego` and `gssapi`)

There are two ways to reach SQL Server:

- **Over the network, port 1433, with a SQL login.** These are the `community.general.mssql_*` modules. They run inside the EE.
- **On the Windows server, with Windows authentication.** These are the `lowlydba.sqlserver` modules: logins, databases, agent jobs, backups. They run on the SQL Server over WinRM and use the **dbatools** PowerShell module, which must be installed on that server.

Only want AD, or only SQL? Remove the other part's lines from `requirements.yml`, `requirements.txt` and `check-list.txt` before you build.

Step 1. Go to the folder

```shell
cd ~/ee/windows
ls
```

You should see

```
ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt
execution-environment.yml krb5-ad-enctypes.conf requirements.txt requirements.yml
```

`ansible.cfg` is missing? Do chapter 2, step 4.

Step 2. Build it

```shell
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-windows:1 context
```

- The first command writes the build recipe into `context/`. The second one builds it.
- The first build takes **10 to 20 minutes**. It downloads everything and compiles the Kerberos libraries.
- `:1` is the version. Your next build is `:2`, and `:1` stays there to go back to.

You should see
at the end, `Successfully tagged localhost/site-ee-windows:1`.

Step 3. Check it

```shell
podman run --rm site-ee-windows:1 ee-check
```

`ee-check` lists every library, collection and tool this EE should have, with `YES` or `no`.

You should see
every line says `YES`, and the last line is `== ALL YES`.

Step 4. Check Kerberos to Active Directory

```shell
podman run --rm -it --network=host site-ee-windows:1 kinit svc_aap@EXAMPLE.MIL
```

Use a real domain account. The domain must be in **capital letters**.

You should see
`Password for svc_aap@EXAMPLE.MIL:`. Kerberos works. Press Ctrl+C, or type the password to get a ticket.

`Cannot find KDC for realm` or `KDC has no support for encryption type`: see chapter 8.

Step 5. Put it into AAP

Do chapter 6 with the image name `site-ee-windows:1` and the AAP name `Windows, AD and SQL EE`.

Step 6. A first job

Below are five small playbooks, one for each part of the EE. Put the ones you want in your project. Make a job template for each, with **this EE**, and run it.

1. Windows, `ee-test-windows.yml`. Inventory: your Windows servers. Credential: your Windows credential.

YAML

```
---
- name: Try the EE - Windows
  hosts: all
  gather_facts: false
  tasks:
    - name: WinRM works
      ansible.windows.win_ping:
```

You should see
each Windows server green, with `ping: pong`.

2. Active Directory, `ee-test-ad.yml`. It runs on a domain controller, or on a server with RSAT. On the template, set **Limit** to one domain controller, for example `dc01.example.mil`, and use the same Windows credential.

YAML

```
---
- name: Try the EE - Active Directory
  hosts: all
  gather_facts: false
  tasks:
    - name: The computer accounts in the Servers OU
      microsoft.ad.object_info:
        ldap_filter: (objectClass=computer)
        search_base: OU=Servers,DC=example,DC=mil
        properties: [dNSHostName, operatingSystem]
      register: ad
    - name: How many
      ansible.builtin.debug:
        msg: "{{ ad.objects | length }} computer accounts"
```

You should see
the number of computer accounts in that OU.

3. SQL Server with a SQL login, `ee-test-mssql.yml`. It runs inside the EE and connects to the server's port 1433.

YAML

```
---
- name: Try the EE - SQL Server with a SQL login
  hosts: localhost
  gather_facts: false
  tasks:
    - name: The databases and their state
      community.general.mssql_script:
        login_host: sql01.example.mil
        login_user: "{{ mssql_user }}"
        login_password: "{{ mssql_password }}"
        db: master
        script: SELECT name, state_desc FROM sys.databases
      register: dbs
    - name: Show them
      ansible.builtin.debug:
        var: dbs.query_results
```

For a first try, give `mssql_user` and `mssql_password` when you launch: on the template, tick **Prompt on launch** next to **Variables**. For real jobs, keep them in a credential. Chapter 4, step 6 shows how to make one.

> **You should see**
> the list of databases, each with `ONLINE`.

4. SQL Server with Windows authentication, `ee-test-sqlwin.yml`. It runs on the SQL Server itself, over WinRM. Put your SQL Servers in an inventory group, here `sqlservers`, and use your Windows credential.

YAML

```
---
- name: Try the EE - SQL Server with Windows authentication
  hosts: sqlservers
  gather_facts: false
  tasks:
    - name: The instance
      lowlydba.sqlserver.instance_info:
        sql_instance: localhost
      register: sql
    - name: Show it
      ansible.builtin.debug:
        var: sql.data
```

It needs **dbatools** on each SQL Server. Run this once, in PowerShell as an administrator: `Install-Module dbatools -Scope AllUsers`. If the servers have no internet, run `Save-Module dbatools -Path C:\temp` on a machine that does. Then copy both folders it makes into `C:\Program Files\WindowsPowerShell\Modules` on each SQL Server.

> **You should see**
> the instance's name, version and edition.

5. MySQL or MariaDB, `ee-test-mysql.yml`. It runs inside the EE and connects to the server's port 3306. Give `db_user` and `db_password` the same way as in playbook 3.

YAML

```
---
- name: Try the EE - MySQL or MariaDB
  hosts: localhost
  gather_facts: false
  tasks:
    - name: The databases
      ansible.mysql.mysql_info:
        login_host: db01.example.mil
        login_user: "{{ db_user }}"
        login_password: "{{ db_password }}"
        filter: [databases, version]
      register: db
    - name: Show the version
      ansible.builtin.debug:
        var: db.version
```

You should see
the server's version, for example `full: 10.11.9-MariaDB`.

Active Directory as an AAP inventory

This EE can also read the computer accounts from AD and turn each one into an AAP host. It uses the `microsoft.ad.ldap` inventory plugin, and nothing is installed on a domain controller.

1. Put a file in your project whose name ends in `microsoft.ad.ldap.yml`, for example `inventories/ad/servers.microsoft.ad.ldap.yml`:

YAML

```
plugin: microsoft.ad.ldap
server: dc01.example.mil          # leave it out and the DCs are found in DNS
tls_mode: ldaps                   # LDAP over TLS, port 636, checked against your CA
auth_protocol: simple             # on a FIPS host: simple over LDAPS, or kerberos - never ntlm
search_base: OU=Servers,DC=example,DC=mil
filter: (operatingSystem=*Server*)
attributes:
  operatingSystem:
    os_name:
keyed_groups:
  - key: os_name
    prefix: os
```

2. Make a credential type for the account. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**, name `AD LDAP bind`.

Input configuration

```
fields:
  - id: username
    type: string
    label: User (user@EXAMPLE.MIL)
  - id: password
    type: string
    label: Password
    secret: true
required: [username, password]
```

Injector configuration

```
env:
  MICROSOFT_AD_LDAP_USERNAME: "{{ username }}"
  MICROSOFT_AD_LDAP_PASSWORD: "{{ password }}"
```

3. Make a credential of that type. A normal, read-only domain account is enough.

4. In your inventory, go to **Sources > Create source**:

- **Source:** *Sourced from a Project*
- **Project:** your project
- **Inventory file:** the file from part 1
- **Credential:** `AD LDAP bind`
- **Execution environment:** `Windows, AD and SQL EE`

Then press **Sync**.

You should see
the sync goes green, and the hosts show up in groups such as `os_Windows_Server_2022_Standard`.

4. EE 2: Network - Cisco and Palo Alto

Image name: `site-ee-network`. Folder: `~/ee/network`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, plus:

For	Collections	Python libraries and tools
Connections and helpers	`ansible.netcommon` (network_cli, netconf, httpapi), `ansible.utils`	`ansible-pylibssh`, `paramiko`, `ncclient`, `netaddr`, `textfsm`, `ttp`, `jmespath`; `ssh`, `sshpass`
Cisco	`cisco.ios` (IOS / IOS-XE), `cisco.nxos` (Nexus), `cisco.asa` (ASA)	(from the connections above)
Palo Alto	`paloaltonetworks.panos` (firewalls and Panorama)	`pan-os-python`, `pan-python`, `panos-upgrade-assurance`, `xmltodict`
The report website on an SMB share		`smbprotocol` (Kerberos through `pyspnego` and `gssapi`)

Cisco is reached over SSH. Palo Alto is reached over HTTPS, through the PAN-OS XML API. Only need one vendor? Remove the other one's lines from `requirements.yml` and `check-list.txt` before you build.

Step 1. Go to the folder

shell

```
cd ~/ee/network
ls
```

You should see
`ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt execution-environment.yml requirements.txt requirements.yml`

Step 2. Build it

shell

```
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-network:1 context
```

This one is quicker, about 5 to 10 minutes. Nothing has to be compiled.

You should see
at the end, `Successfully tagged localhost/site-ee-network:1`.

Step 3. Check it

```shell
podman run --rm site-ee-network:1 ee-check
```

> **You should see**
> every line says `YES`, and the last line is `== ALL YES`.

Step 4. Put it into AAP

Do chapter 6 with the image name `site-ee-network:1` and the AAP name `Network EE (Cisco and Palo Alto)`.

Step 5. Cisco: a first job

1. The inventory. Put your Cisco devices in a group, for example `ios`, and give the group these variables. In AAP: the inventory **> Groups > ios > Variables**.

```yaml
ansible_connection: ansible.netcommon.network_cli
ansible_network_os: cisco.ios.ios
ansible_network_cli_ssh_type: libssh
ansible_become: true
ansible_become_method: enable
```

Nexus: `ansible_network_os: cisco.nxos.nxos`. ASA: `ansible_network_os: cisco.asa.asa`.

2. The credential. Use a *Machine* credential: the username, the password, and the enable password as the **Privilege Escalation Password**.

3. The playbook, `ee-test-cisco.yml`:

```yaml
---
- name: Try the network EE - Cisco
  hosts: ios
  gather_facts: false
  tasks:
    - name: Facts
      cisco.ios.ios_facts:
        gather_subset: min
    - name: What it runs
      ansible.builtin.debug:
        msg: "{{ ansible_net_hostname }} runs {{ ansible_net_version }}"
```

> **You should see**
> each device green, with its name and IOS version.

Step 6. Palo Alto: a first job

The Palo Alto modules run inside the EE and talk to the firewall over HTTPS. They take the login from a `provider` setting, not from a Machine credential, so you make a small credential type for it.

1. The credential type. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**, name `Palo Alto PAN-OS`.

Input configuration

```
fields:
  - id: username
    type: string
    label: PAN-OS user
  - id: password
    type: string
    label: Password
    secret: true
required: [username, password]
```

Injector configuration

```
extra_vars:
  panos_username: "{{ username }}"
  panos_password: "{{ password }}"
```

Then make a credential of that type, with a PAN-OS admin account. A read-only admin is enough for this test.

2. The inventory. Put the firewalls (and Panorama) in a group, for example `panos`, with these group variables:

YAML

```
ansible_connection: local
ansible_python_interpreter: "{{ ansible_playbook_python }}"
```

3. The playbook, `ee-test-panos.yml`:

YAML

```
---
- name: Try the network EE - Palo Alto
  hosts: panos
  gather_facts: false
  tasks:
    - name: System facts
      paloaltonetworks.panos.panos_facts:
        provider:
          ip_address: "{{ ansible_host | default(inventory_hostname) }}"
          username: "{{ panos_username }}"
          password: "{{ panos_password }}"
        gather_subset: [system]
    - name: What it runs
      ansible.builtin.debug:
        msg: "{{ ansible_net_hostname }} runs PAN-OS {{ ansible_net_version }}"
```

On the job template, choose the `Palo Alto PAN-OS` credential, not a Machine credential.

> **You should see**
> each firewall green, with its name and PAN-OS version.

`certificate verify failed`: the firewall's HTTPS certificate is not from a CA in `ca/`. See chapter 8.

5. EE 3: All-in-one

Image name: `site-ee-all`. Folder: `~/ee/all-in-one`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, everything in EE 1 and EE 2, plus:

For	Collections	Python libraries
Linux	`ansible.posix`, `ansible.utils`, `community.general`, `community.crypto`, `containers.podman`, `redhat.rhel_system_roles`	`jmespath`, `netaddr`, `dnspython`, `cryptography`
VMware	`vmware.vmware`	`pyVmomi`, `vmware-vcenter` (the vSphere Automation SDK)
NetApp	`netapp.ontap`	`requests`, `solidfire-sdk-python` and the others the collection lists
ServiceNow	`servicenow.itsm` (tickets, and the CMDB as an AAP inventory)	`requests`
Times in your time zone		`tzdata` (the system package; `report_time_zone`)

This is the EE for the site-automation jobs. `redhat.rhel_system_roles` is only on Red Hat's Automation Hub. If your hub does not have it, remove its line from `requirements.yml` and `check-list.txt`.

Built the all-in-one EE before?
If you built `site-ee-all:1` from the earlier EE guide, this version adds Palo Alto, `lowlydba.sqlserver`, `smbprotocol` (the report website on an SMB share) and `tzdata`. Build it as `site-ee-all:2` instead of `:1` in the commands below, so `:1` stays there to go back to.

Step 1. Go to the folder

shell

```
cd ~/ee/all-in-one
ls
```

You should see
`ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt execution-environment.yml krb5-ad-enctypes.conf requirements.txt requirements.yml`

Step 2. Build it

```shell
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-all:1 context
```

This is the biggest build, about **15 to 25 minutes**.

> **You should see**
> at the end, `Successfully tagged localhost/site-ee-all:1`.

Step 3. Check it

```shell
podman run --rm site-ee-all:1 ee-check
podman run --rm -it --network=host site-ee-all:1 kinit svc_aap@EXAMPLE.MIL
```

> **You should see**
> `== ALL YES` at the end of the first command. The second one shows `Password for svc_aap@EXAMPLE.MIL:`; press Ctrl+C.

Step 4. Put it into AAP

Do chapter 6 with the image name `site-ee-all:1` and the AAP name `All-in-one EE`.

Step 5. A first job

Every first job of EE 1 and EE 2 works with this EE too. For site-automation, run a template you know with the **All-in-one EE**, for example *Health check* in Check mode.

> **You should see**
> the job is green, just as it was with its old EE.

6. Put an EE into AAP

Do this chapter once for each EE you built. The examples use `site-ee-windows:1`. For the other EEs, change the name.

Step 1. Put the image where AAP can pull it

With a registry (a Private Automation Hub has one):

```shell
podman login aap.example.mil
podman tag site-ee-windows:1 aap.example.mil/site-ee-windows:1
podman push aap.example.mil/site-ee-windows:1
```

You should see
`Writing manifest to image destination`, and no error.

Without a registry: save the image as a file, and load it on **every AAP execution node**, as the user AAP runs jobs as:

```shell
podman save -o site-ee-windows-1.tar site-ee-windows:1    # on the build machine
podman load -i site-ee-windows-1.tar                      # on each execution node
```

You should see
`Loaded image: localhost/site-ee-windows:1` on each node.

Step 2. Add it in AAP

Go to **Automation Execution > Infrastructure > Execution Environments > Create execution environment**:

Field	With a registry	Loaded from a file
Name	`Windows, AD and SQL EE`	`Windows, AD and SQL EE`
Image	`aap.example.mil/site-ee-windows:1`	`localhost/site-ee-windows:1`
Pull	*Only pull the image if not present before running*	*Never pull container before running*
Registry credential	your hub's registry credential	none

Press **Create execution environment**.

Step 3. Use it

Open a job template, choose the EE under **Execution environment**, and save. You can also choose it on an inventory source, or set it as your organization's default.

Job templates for	Use
Windows servers, Active Directory, SQL Server, MySQL, MariaDB	`Windows, AD and SQL EE`
Cisco switches, routers, ASA; Palo Alto firewalls, Panorama	`Network EE (Cisco and Palo Alto)`
site-automation, Linux, VMware, NetApp, ServiceNow, or a mix	`All-in-one EE`

You should see
the job's **Details** tab shows the new EE, and the first job (chapters 3 to 5) is green.

7. Keep them up to date

Pin the versions after the first good build

A new build downloads the newest collections, so two builds can differ. Once a build works, pin each collection to the version it got. `ee-check` shows the versions:

```shell
podman run --rm site-ee-windows:1 ee-check
```

Then write each version into `requirements.yml`, for example:

```YAML
  - name: microsoft.ad
    version: "==1.12.1"
```

Rebuild with a new tag

1. Change the files: a collection added, a version pinned or raised.
2. Build with the **next tag**, for example `-t site-ee-windows:2` (step 2 of the EE's chapter).
3. Check it with `ee-check`, then push or load it (chapter 6, step 1).
4. In AAP, change the EE's **Image** to `:2`. If a job breaks, change it back to `:1`.

Rebuild every few months, even without changes. That is how the Python libraries get their fixes.

Why three collections are pinned

`requirements.yml` pins three collections. Their newest versions need a newer ansible-core than 2.16, and `base-ee:1.0` may have 2.16:

```YAML
  - name: community.general
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: community.crypto
    version: "<3.0.0"       # 2.x: the newest for ansible-core 2.16 (3 needs 2.17)
  - name: lowlydba.sqlserver
    version: "<3.0.0"       # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
```

If `ansible --version` (chapter 2, step 3) shows 2.20, you can remove those `version:` lines. Then build the next tag to get the newest versions.

base-ee:1.0 already has some of these collections

If `base-ee:1.0` already has a collection from `requirements.yml`, the build keeps the version it has. The build upgrades it only when another collection needs a newer one: `community.windows` needs `ansible.windows` 3 or newer, for example. `ee-check` shows the versions you end up with.

A new base EE

If you build a new base EE, for example `base-ee:1.1`, change the `name:` line in the three `execution-environment.yml` files. Then build the next tag of each EE.

8. When something goes wrong

You see	It means	Do
`localhost/base-ee:1.0: image not known`, or `pull access denied` for `base-ee`	the build machine does not have your base EE	chapter 2, step 3: pull or load it, and check the `name:` line
`Missing Ansible or Ansible Runner for selected Python`	the build looks for Ansible on another Python than the base EE's	chapter 2, step 3: *The Python must match*
`unauthorized` pulling `aap.example.mil/base-ee:1.0` or `registry.redhat.io/...`	podman is not logged in to that registry	`podman login aap.example.mil` (or `registry.redhat.io`)
`pasta ... permission denied` or `netavark` errors	rootless podman's network helper is blocked	keep `--network=host` on `podman build`
`additional_build_files ... ansible.cfg ... does not exist`	`ansible.cfg` is not in this folder	chapter 2, step 4
`HTTP Code: 401` or `403` installing collections	no token, or the wrong one	`read -rs TOKEN` again (chapter 2, step 5); a new token: *Automation Content > API token*
`HTTP Code: 401` from `galaxy.ansible.com`	a token was sent to Galaxy	in `ansible.cfg`, name the Galaxy server `galaxy` (chapter 2, step 4)
`Failed to find collection ...`	your hub does not have that collection	ask the hub admin to sync it (chapter 2, step 4), or remove its line from `requirements.yml` and `check-list.txt`
`No match for argument: <package>`	that RHEL package is not in the build machine's repositories	check the subscription (`sudo subscription-manager status`); if a collection wants it only to compile, add it under `exclude: system:` in `execution-environment.yml`
`Could not find a version that satisfies the requirement ...`	the build cannot reach PyPI	add `--build-arg PIP_INDEX_URL=...` (chapter 2, step 7)
`Python.h: No such file or directory`	the C headers of the EE's Python are missing	the build installs them from your RHEL repositories: check the subscription
`ee-check` ends with `MISSING`	a library, collection or tool is not in the image	read the build's output for that name; rebuild
a job: `couldn't resolve module/action`	the module's collection is not in the job's EE	the template's **Execution environment**; `ee-check` lists the collections
a job: `Collection ... does not support Ansible version`	the collection needs a newer ansible-core than `base-ee:1.0` has	pin it to an older version: chapter 7, *Why three collections are pinned*

You see	It means	Do
a job: `certificate verify failed`	the EE does not have your CA	put it in `ca/` (chapter 2, step 6), and build the next tag
`kinit: Cannot find KDC for realm "EXAMPLE.MIL"`	the EE cannot find the domain controllers in DNS	use `--network=host` on `podman run`; the domain must be in capitals
a job: `KDC has no support for encryption type`	the FIPS Kerberos setting is not in the image	`ee-check` must say YES to both Kerberos lines: rebuild from these files
a job: `ntlm: ... unsupported`	NTLM on a FIPS host (no MD4)	use Kerberos: the user as `user@EXAMPLE.MIL`
Palo Alto: `Failed connection: URLError` or a timeout	the EE cannot reach the firewall's HTTPS port (443)	firewall rules from the AAP execution nodes to the management interface
Palo Alto: `Invalid credentials`	the user or password is wrong, or the account has no XML API rights	the admin role must allow the XML API
Cisco: `unable to open shell` or `Authentication failed`	SSH or the login failed	test from the EE: `podman run --rm -it --network=host site-ee-network:1 ssh user@switch01`
`lowlydba.sqlserver`: `dbatools` not found	dbatools is not on that SQL Server	chapter 3, step 6: install dbatools

9. Appendix: the build files

The files in the repository's `ee/` folder (and in every release zip), one folder per EE. **Copy them from there**, not from this PDF: a PDF copy can change spaces, and YAML depends on them.

EE 1: Windows, AD and SQL - folder windows/

windows/execution-environment.yml

```
---
# The Windows, Active Directory and SQL execution environment: Windows over WinRM (Kerberos),
  Active
# Directory (and AD as an AAP inventory), SQL Server, MySQL and MariaDB.
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-windows:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon (your base EE may have it) asks for these to compile ansible-pylibssh,
        which
      # comes ready-built (a wheel); both are in CodeReady Builder, which a RHEL build does not
        have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: krb5-ad-enctypes.conf
    dest: configs
  - src: check-ee.sh
    dest: configs
  - src: check-list.txt
```

windows/execution-environment.yml (continued)

```
    dest: configs
  - src: ca
    dest: ca

additional_build_steps:
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # a PyPI mirror, when the build host cannot reach pypi.org: --build-arg
      PIP_INDEX_URL=https://...
    - ARG PIP_INDEX_URL
    # the C headers of the EE's own Python (gssapi, krb5 and pykerberos compile against them), and
      pip's
    # wheel package: pykerberos (WinRM's Kerberos) is then built here, not in the final image
    - RUN $PKGMGR install -y python$($PYCMD -c 'import sys; print("%d.%d" %
      sys.version_info[:2])')-devel && $PKGMGR clean all
    - RUN $PYCMD -m pip install --no-cache-dir wheel
  append_final:
    # Kerberos to Active Directory from a FIPS host: AD's AES-SHA1 keys allowed
      (krb5-ad-enctypes.conf)
    - COPY _build/configs/krb5-ad-enctypes.conf /etc/krb5.conf.d/ad-enctypes.conf
    - RUN rm -f /etc/krb5.conf.d/crypto-policies
    # Kerberos tickets in a file: the kernel keyring RHEL uses by default does not work in a
      container
    - ENV KRB5CCNAME=FILE:/tmp/krb5cc_ee
    # your CA certificates (ca/*.pem) in the trust store - LDAPS, SQL Server TLS, AAP
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    - RUN chmod 755 /usr/local/bin/ee-check
```

windows/requirements.yml

```
---
# The collections of the Windows, Active Directory and SQL EE. "certified" = Red Hat Automation
  Hub (also in a
# Private Automation Hub that syncs it); "community" = Galaxy (a Private Automation Hub needs its
  community
# repository to have it). ansible-builder reads each collection's own Python and system needs.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
collections:
  # ---- Windows ----
  - name: ansible.windows               # certified: the core Windows modules (win_service,
    win_updates ...)
  - name: community.windows             # community: more Windows modules
  # ---- Active Directory ----
  - name: microsoft.ad                  # certified: users, groups, computers, OUs - and AD as an
    AAP inventory
  # ---- SQL Server ----
  - name: community.general             # community: mssql_db, mssql_script (SQL Server over port
    1433)
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: lowlydba.sqlserver            # community: SQL Server through dbatools on a Windows host
    (logins, jobs, backups)
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
  # ---- MySQL and MariaDB ----
  - name: ansible.mysql                 # MySQL and MariaDB (was community.mysql)
```

windows/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs). They
  come from PyPI.
# ---- Windows over WinRM ----
pywinrm[kerberos]
requests-kerberos
requests-ntlm          # NTLM: not on a FIPS host (no MD4); Kerberos is the way there
pypsrp[kerberos]       # the psrp connection, an alternative to winrm
# ---- Active Directory over LDAP, from the EE (microsoft.ad brings sansldap, pyspnego, dnspython)
  ----
ldap3                  # pure Python: your own LDAP scripts
# ---- SQL Server (community.general.mssql_db, mssql_script) - ansible.mysql brings PyMySQL itself
  ----
pymssql
# ---- general ----
jmespath               # the json_query filter
dnspython              # the community.general.dig lookup
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

windows/bindep.txt

```
# System packages (RPMs, from Red Hat) the collections do not bring themselves (ansible-builder
  adds theirs).
krb5-workstation [platform:rpm]       # kinit / klist: check Kerberos inside the EE
bind-utils [platform:rpm]             # dig / nslookup: check DNS (the domain controllers' SRV
  records)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
```

windows/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name Windows, Active Directory and SQL EE
# ---- Windows over WinRM ----
lib winrm pywinrm
lib kerberos pykerberos
lib requests_kerberos requests-kerberos
lib requests_ntlm requests-ntlm
lib pypsrp pypsrp
# ---- Active Directory ----
lib spnego pyspnego
lib gssapi gssapi
lib krb5 krb5
lib sansldap sansldap
lib ldap3 ldap3
lib dns dnspython
# ---- SQL ----
lib pymssql pymssql
lib pymysql PyMySQL
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
lib cryptography cryptography
collection ansible.windows
collection community.windows
collection microsoft.ad
collection community.general
collection lowlydba.sqlserver
collection ansible.mysql
tool kinit
tool klist
tool dig
tool ping
kerberos
```

EE 2: Network (Cisco and Palo Alto) - folder network/

network/execution-environment.yml

```
---
# The network execution environment: Cisco (IOS / IOS-XE, Nexus, ASA) and Palo Alto (PAN-OS
  firewalls and
# Panorama).
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-network:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon asks for these to compile ansible-pylibssh, which comes ready-built (a
        wheel);
      # both are in the CodeReady Builder repository, which a RHEL build does not have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: check-ee.sh
    dest: configs
  - src: check-list.txt
    dest: configs
  - src: ca
    dest: ca
```

network/execution-environment.yml (continued)

```
additional_build_steps:
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # a PyPI mirror, when the build host cannot reach pypi.org: --build-arg
      PIP_INDEX_URL=https://...
    - ARG PIP_INDEX_URL
    - RUN $PYCMD -m pip install --no-cache-dir wheel
  append_final:
    # your CA certificates (ca/*.pem) in the trust store - Panorama, the firewalls' HTTPS, AAP
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    - RUN chmod 755 /usr/local/bin/ee-check
```

network/requirements.yml

```
---
# The collections of the network EE. "certified" = Red Hat Automation Hub (also in a Private
  Automation Hub
# that syncs it); "partner" = made by the vendor (in your hub's rh-certified repository when it
  has it -
# otherwise Galaxy). ansible-builder reads each collection's own Python and system needs.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
# Only one vendor? Remove the other's lines here and in check-list.txt.
collections:
  # ---- the network connections, and helpers ----
  - name: ansible.netcommon             # certified: network_cli (SSH), netconf, httpapi
    connections
  - name: ansible.utils                 # certified: ipaddr filters, cli_parse, validate
  # ---- Cisco ----
  - name: cisco.ios                     # certified: IOS / IOS-XE
  - name: cisco.nxos                    # certified: Nexus
  - name: cisco.asa                     # certified: ASA firewalls
  # ---- Palo Alto ----
  - name: paloaltonetworks.panos        # partner: PAN-OS firewalls and Panorama (over their XML
    API, HTTPS)
```

network/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs). They
  come from PyPI.
# The collections bring the rest: ansible-pylibssh, paramiko, ncclient (Cisco); pan-os-python,
  pan-python,
# xmltodict (Palo Alto); netaddr, textfsm, ttp (ansible.utils).
jmespath               # the json_query filter
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

network/bindep.txt

```
# System packages (RPMs, from Red Hat) the collections do not bring themselves (ansible-builder
  adds theirs).
openssh-clients [platform:rpm]        # ssh / scp: test a device's SSH from inside the EE
sshpass [platform:rpm]                # password SSH (network devices that need it)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
bind-utils [platform:rpm]             # dig / nslookup
```

network/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name Network EE (Cisco and Palo Alto)
# ---- Cisco and the network connections ----
lib pylibsshext ansible-pylibssh
lib paramiko paramiko
lib ncclient ncclient
lib netaddr netaddr
lib textfsm textfsm
lib ttp ttp
# ---- Palo Alto ----
lib panos pan-os-python
lib pan.xapi pan-python
lib panos_upgrade_assurance panos-upgrade-assurance
lib xmltodict xmltodict
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
collection ansible.netcommon
collection ansible.utils
collection cisco.ios
collection cisco.nxos
collection cisco.asa
collection paloaltonetworks.panos
tool ssh
tool sshpass
tool ping
tool dig
```

EE 3: All-in-one - folder all-in-one/

all-in-one/execution-environment.yml

```
---
# The all-in-one execution environment: Linux, Windows and Active Directory, VMware, NetApp,
  databases
# (MySQL, MariaDB, SQL Server), Cisco, Palo Alto, ServiceNow - and every site-automation job.
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-all:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon asks for these to compile ansible-pylibssh, which comes ready-built (a
        wheel);
      # both are in the CodeReady Builder repository, which a RHEL build does not have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: krb5-ad-enctypes.conf
    dest: configs
  - src: check-ee.sh
    dest: configs
  - src: check-list.txt
    dest: configs
```

all-in-one/execution-environment.yml (continued)

```
  - src: ca
    dest: ca

additional_build_steps:
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # a PyPI mirror, when the build host cannot reach pypi.org: --build-arg
      PIP_INDEX_URL=https://...
    - ARG PIP_INDEX_URL
    # the C headers of the EE's own Python (gssapi, krb5 and pykerberos compile against them), and
      pip's
    # wheel package: pykerberos (WinRM's Kerberos) is then built here, not in the image, which has
      no compiler
    - RUN $PKGMGR install -y python$($PYCMD -c 'import sys; print("%d.%d" %
      sys.version_info[:2])')-devel && $PKGMGR clean all
    - RUN $PYCMD -m pip install --no-cache-dir wheel
  append_final:
    # Kerberos to Active Directory from a FIPS host: AD's AES-SHA1 keys allowed
      (krb5-ad-enctypes.conf)
    - COPY _build/configs/krb5-ad-enctypes.conf /etc/krb5.conf.d/ad-enctypes.conf
    - RUN rm -f /etc/krb5.conf.d/crypto-policies
    # Kerberos tickets in a file: the kernel keyring RHEL uses by default does not work in a
      container
    - ENV KRB5CCNAME=FILE:/tmp/krb5cc_ee
    # your CA certificates (ca/*.pem) in the trust store - vCenter, ServiceNow, AAP, LDAPS, ONTAP,
      Panorama
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    - RUN chmod 755 /usr/local/bin/ee-check
```

all-in-one/requirements.yml

```
---
# The Ansible collections of the all-in-one execution environment (ee/all-in-one,
  docs/EXECUTION_ENVIRONMENT.md).
# "certified" = Red Hat Automation Hub (also in a Private Automation Hub that syncs it);
  "community" = Galaxy;
# "partner" = made by the vendor (in your hub's rh-certified repository when it has it - otherwise
  Galaxy)
# (a Private Automation Hub needs its community repository synced for these). ansible-builder
  reads each
# collection's own Python and system needs: they are not repeated in requirements.txt /
  bindep.txt.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
collections:
  # ---- Linux ----
  - name: ansible.posix                # certified: acl, firewalld, mount, sysctl, selinux ...
  - name: ansible.utils                # certified: ipaddr filters, validate, cli_parse
  - name: community.general            # community: hundreds of modules (ldap_*, mssql_*, nmcli,
    ...)
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: community.crypto             # community: certificates, CSRs, keys
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.17)
  - name: containers.podman            # community: podman containers, pods, Quadlet
  - name: redhat.rhel_system_roles     # certified (Automation Hub only): RHEL System Roles
  # ---- Windows and Active Directory ----
  - name: ansible.windows              # certified: the core Windows modules
  - name: microsoft.ad                 # certified: AD objects, users, groups - and the LDAP
    inventory plugin
  - name: community.windows            # community: more Windows modules
  # ---- VMware ----
  - name: vmware.vmware                # certified: vCenter (site-automation's VMware jobs need
    >= 2.11)
    version: ">=2.11.0"
  # ---- NetApp ----
  - name: netapp.ontap                 # certified: ONTAP REST modules
  # ---- Databases ----
  - name: ansible.mysql                # certified: MySQL and MariaDB (was community.mysql)
  - name: lowlydba.sqlserver           # community: SQL Server through dbatools on a Windows host
    (logins, jobs, backups)
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
  # ---- Cisco and network ----
  - name: ansible.netcommon            # certified: network_cli, netconf, httpapi connections
  - name: cisco.ios                    # certified: IOS / IOS-XE
  - name: cisco.nxos                   # certified: Nexus
  - name: cisco.asa                    # certified: ASA firewalls
  # ---- Palo Alto ----
  - name: paloaltonetworks.panos       # partner: PAN-OS firewalls and Panorama (over their XML
    API, HTTPS)
  # ---- ServiceNow ----
```

all-in-one/requirements.yml (continued)

```
  - name: servicenow.itsm              # certified: tickets, and the CMDB inventory plugin
    (servicenow.itsm.now)
```

all-in-one/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs).
# Windows over WinRM (ansible.windows, community.windows, microsoft.ad modules, site-automation's
  Windows jobs)
pywinrm[kerberos]
requests-kerberos
requests-ntlm          # NTLM: not on a FIPS host (no MD4); Kerberos is the way there
pypsrp[kerberos]       # the psrp connection, an alternative to winrm
# Active Directory from the EE itself, over LDAP (microsoft.ad brings sansldap, pyspnego,
  dnspython)
ldap3                  # pure Python: your own scripts, no compiler
# SQL Server (community.general.mssql_db, mssql_script)
pymssql
# Linux and general
jmespath               # the json_query filter
dnspython              # the community.general.dig lookup
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

all-in-one/bindep.txt

```
# System packages (RPMs) the collections do not bring themselves (ansible-builder adds theirs).
# [compile] = only while building; the rest stays in the image.
krb5-workstation [platform:rpm]       # kinit / klist: check Kerberos inside the EE
openssh-clients [platform:rpm]
sshpass [platform:rpm]                # password SSH (network devices that need it)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
bind-utils [platform:rpm]             # dig / nslookup: check DNS (the domain controllers' SRV
  records)
tzdata [platform:rpm]                 # the time zones: report_time_zone (0.32.0), the VMware
  reports in your time zone
```

all-in-one/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name All-in-one EE
# ---- Windows over WinRM ----
lib winrm pywinrm
lib kerberos pykerberos
lib requests_kerberos requests-kerberos
lib requests_ntlm requests-ntlm
lib pypsrp pypsrp
# ---- Active Directory ----
lib spnego pyspnego
lib gssapi gssapi
lib krb5 krb5
lib sansldap sansldap
lib ldap3 ldap3
lib dns dnspython
# ---- VMware ----
lib pyVmomi pyVmomi
lib com.vmware.vcenter_client vmware-vcenter
# ---- NetApp, ServiceNow ----
lib requests requests
lib solidfire solidfire-sdk-python
# ---- SQL ----
lib pymssql pymssql
lib pymysql PyMySQL
# ---- Cisco and the network connections ----
lib pylibsshext ansible-pylibssh
lib paramiko paramiko
lib ncclient ncclient
lib netaddr netaddr
lib textfsm textfsm
# ---- Palo Alto ----
lib panos pan-os-python
lib pan.xapi pan-python
lib xmltodict xmltodict
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
lib cryptography cryptography
collection ansible.posix
collection ansible.utils
collection community.general
collection community.crypto
collection containers.podman
collection redhat.rhel_system_roles
collection ansible.windows
collection microsoft.ad
collection community.windows
```

all-in-one/check-list.txt (continued)

```
collection vmware.vmware
collection netapp.ontap
collection ansible.mysql
collection lowlydba.sqlserver
collection ansible.netcommon
collection cisco.ios
collection cisco.nxos
collection cisco.asa
collection paloaltonetworks.panos
collection servicenow.itsm
tool kinit
tool klist
tool ssh
tool sshpass
tool dig
tool ping
kerberos
```

The same in every folder

ansible.cfg.example (in all three folders)

```
# Copy to ansible.cfg (this folder) when the collections come from your automation hub. The URLs:
  AAP >
# Automation Content > Repositories > a repository > "Copy CLI configuration" (AAP 2.5 and later:
  the
# platform's own address). The token is NOT written here: it is given at build time (--build-arg).
[galaxy]
server_list = rh_certified, community, published

[galaxy_server.rh_certified]
url = https://aap.example.mil/api/galaxy/content/rh-certified/

[galaxy_server.community]
url = https://aap.example.mil/api/galaxy/content/community/

[galaxy_server.published]
url = https://aap.example.mil/api/galaxy/content/published/
```

check-ee.sh (in all three folders)

```
#!/bin/bash
# ee-check: does this execution environment have what it should? Run it inside the image:
#   podman run --rm <image> ee-check
# What it looks for is in /usr/local/share/ee-check.list (the check-list.txt of the EE's folder).
# It ends with "ALL YES" - or with what is missing (and exit code 1).
list=/usr/local/share/ee-check.list
missing=0
yes() { echo "  YES  $1"; }
no()  { echo "  no   $1"; missing=$((missing + 1)); }
# the Python Ansible runs on (the first line of the ansible command), not the system's python3
py=$(head -1 "$(command -v ansible)" 2>/dev/null | sed -e 's/^#! *//' -e 's/ .*//')
[ -x "$py" ] || py=python3
echo "== $(sed -n 's/^name //p' $list)"
echo "== Python: $($py --version 2>&1)   ansible-core: $(ansible --version 2>/dev/null | head -1)"
echo "== Python libraries"
while read -r kind a b; do
  [ "$kind" = lib ] || continue
  if $py -c "import $a" 2>/dev/null; then yes "$b"; else no "$b"; fi
done < $list
echo "== Collections"
have=$(ansible-galaxy collection list 2>/dev/null)
while read -r kind a _; do
  [ "$kind" = collection ] || continue
  v=$(echo "$have" | awk -v c="$a" '$1 == c { print $2; exit }')
  if [ -n "$v" ]; then yes "$a $v"; else no "$a"; fi
done < $list
echo "== Tools"
while read -r kind a _; do
  [ "$kind" = tool ] || continue
  if command -v "$a" >/dev/null; then yes "$a"; else no "$a"; fi
done < $list
if grep -q '^kerberos' $list; then
  echo "== Kerberos (FIPS, Active Directory)"
  if [ -e /etc/krb5.conf.d/crypto-policies ]; then no "crypto-policies link removed (AD's AES-SHA1
    keys are refused on a FIPS host)"
  else yes "crypto-policies link removed"; fi
  if grep -qs 'aes256-sha1' /etc/krb5.conf.d/ad-enctypes.conf; then yes "AD encryption types
    allowed (ad-enctypes.conf)"
  else no "AD encryption types allowed (ad-enctypes.conf)"; fi
fi
echo "== Your CA certificates added: $(ls /etc/pki/ca-trust/source/anchors/ 2>/dev/null | wc -l)"
if [ "$missing" -eq 0 ]; then echo "== ALL YES"; else echo "== $missing MISSING - see the lines
  with 'no'"; exit 1; fi
```

krb5-ad-enctypes.conf (in windows and all-in-one)

```
# Kerberos to Active Directory from a FIPS host (STIG). The EE's Kerberos would follow the host's
  FIPS
# policy and offer only AES with SHA-2, while Active Directory's keys are AES with SHA-1: "KDC has
  no
# support for encryption type". This allows both AES kinds - never RC4 or DES.
[libdefaults]
  permitted_enctypes = aes256-sha2 aes128-sha2 aes256-sha1 aes128-sha1
```